STRATEGIES TO IMPLEMENT AN ALGORITHMIC C++ 15 PUZZLES PROGRAM.

Jennifer R.Deib

School of Electrical & Information Engineering, University of the Witwatersrand, Private Bag 3, 2050, Johannesburg, South Africa

Abstract:

15 puzzle is a classical one-player sliding puzzle game that allows the re-arrangement of an unshuffled puzzle in accordance with horizontal and vertical movement of a 4x4 board. The games' goal is to re-arrange the puzzle pieces in numerical order and have the blank tile in the last index of the rows and columns. The puzzle game has been modified into a computer program through C++ implementation. The computer program differs from the original game as it does not require human input but rather compiles using the IDA* algorithm with the Manhattan distance heuristic. The program design is discussed thoroughly in this report with significant reasons and support. It is assumed that the reader has some sense of knowledge of the C++ language.

Keywords: algorithm, C++, heuristic, IDA*, Manhattan, puzzle

1. Introduction

The aim of the project was to create and implement a program using the C++ language that can solve any solvable puzzle configuration. "15 puzzles" is also known as the mystic square and recognized globally by several different names and for its pondering solvability debates. The objective of the game is trivial, a 4x4 matrix has 15 tiles that are represented by numerical values and a blank position ('-1' in this instance) must be rearranged in numerical order with the blank tile situated at the last position which is at index (3,3) in terms of an array. The only limitation is that the numerical tiles can only swap with the blank tile. The usual requirement of solving the game is gained by user input into a computer server or moving the wooden blocks around in a physical realm [1]. However, in this program user-input has been restricted and was replaced by a well-known search algorithm. The game has multiple restrictions and constraints which are touched on in section 2 thus leading to further explanations of design and implementation of the algorithm in section 3.

2. Problem Definition:

2.1 Gameplay:

The program initially starts by reading a random, shuffled puzzle from the input text file and determines if a puzzle is solvable or not. If the puzzle is solvable the puzzle will be solved with the use of the IDA* algorithm that makes use of critical mathematical logic and equations to find a path that will achieve the lowest possible number of moves to solve any solvable configuration. Due to the use of an algorithm, user input is not required to solve the game. The algorithm in use can only make movements in horizontal or vertical directions to arrange the puzzle pieces in numerical order in accordance with the blank tile. The output text file contains results of how the puzzle in question was solved and is described under the assumptions heading in 2.2.3.

2.2. Specifications, constraints and assumptions:

2.2.1 Specifications of code implementation:

- Receive and read an undetermined amount of game boards (puzzles) from an input text file.
- Create an algorithm that solves each board by determining which is the best move to make for each round so that moves are limited and efficient.
- Make use of a user defined class to organize user defined functions.
- The code needs to compile and run with specified outputs.

2.2.2 Constraints:

- The usage of global variables is restricted.
- The amount of moves to solve the game is limited to 1000 moves.
- Only horizontal and vertical movement can be used to solve the puzzle.
- The board size is limited to the 4x4 matrix for each puzzle.

- The use of console input from a user is restricted.
- The output must be displayed to an output file named “output.txt”.
- Arrays are less dynamic compared to vectors and is used mostly in this program which can lead to inefficient memory utilization and allocation.
- The use of vectors to store moves can use up more memory space and cause the program to run slower as vectors do not have a set size and are dynamic.

2.2.3 Assumptions:

- A game will end in two ways either solves the puzzle within 1000 moves or reaches the maximum move limit and does not solve.
- A puzzle is either solvable or unsolvable, hence if found to be unsolvable the program will move onto the next solvable puzzle.
- The algorithm will use the best possible move as a calculation of the shortest distance (Manhattan distance) has been made for each move.
- The game will always start by solving puzzle one and then move onto the next.
- The output will give all details regarding the puzzle number, the solvability, the number of moves and what moves were made in terms of index position and characters.

2.3 The success criteria:

The program should compile and run in such a way that it displays the desired output in relation to all specifications and assumptions listed, hence meeting all the requirements of the code and delivering promising results that match those in the mind of what is sought to be a success. This success criteria was further evaluated in Section 5 followed by recommendations for possible improvements and adjustments that meet the most optimal level by acknowledging setbacks and limitations that omit the process of achieving the most efficient results.

3.Design and implementation:

The design of the code began with planning that came to acknowledge that the most suitable way to get an understandable and more organized code is with the use of classes. The only suitable way to achieve a successful result was to implement object-oriented programming.
Hence, this program makes use of two classes namely the class to control the functions of the state and actions of a puzzle board and a class that purely deals with the algorithm of the code. Therefore, the design will consist of five files namely: main.cpp, puzzleBoard.h, puzzleBoard.cpp, Algorithm.h and Algorithm.cpp. A flowchart that shows that interrelation between the classes and the main file is found in the appendix section 3, figure 1.

3.1 puzzleBoard Class Design:

The *puzzleBoard* class was implemented for the board functionality and general features of the game. Figure 1.1 indicates the name of the class and includes what the public and private sections entail.

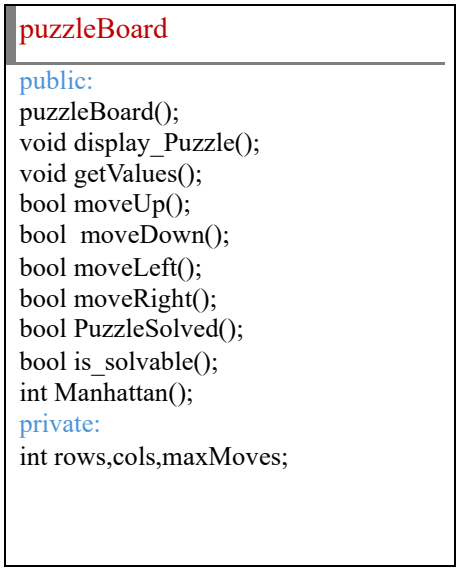

Figure1.1. Layout of the puzzle board class

3.1.1 puzzleBoard.h file contents and deliverables:
The puzzle board header file contains all the declarations in both a public and private section for variables and user-defined functions contained in the puzzleBoard.cpp. The private section consists of variables to avoid the use of global variables and the public section contains function declarations that aid in the functionality of the board structure and state of the game. Section 3.1.2 discusses in depth tasks and responsibilities of each declaration.

3.1.2 puzzleBoard.cpp functionality and support:
The file begins with an empty constructor to set the basis of the class.
“getValues’ allows the variables in the private section to be returned and in use by the client code.

"display_Puzzle" is not required, however, was used for debugging purposes to ensure the correct functionality of each of the basic features of the board.
There are four functions that ensure the correct vertical and horizontal motion of the tiles and consists of row and column boundary conditions to ensure the validity of the movement. The standard swap function was used to make the move by swapping the rows to achieve vertical motion and swapping columns to achieve horizontal motion.
"PuzzleSolved" runs through a nested for loop to verify that the tiles are in numerical order by incrementing though each row and column and ensuring the blank position is situated in the final position of the array. This aids in determining whether the board is in a solved state.
"is_solvable" is an additional feature to the board that determines whether the configuration is solvable or not. A calculation for the number of inversions was made (inversions refer tiles that are in reverse order of their solved state). This calculation is achieved by running through the board and comparing tiles according to the conditions of placement. The blank position is found through a loop and compared with the inversions to determine whether certain conditions are met. If the blank position from the bottom row is odd the inversions must be even and if in and if in an even row from the bottom must have an odd number of inversions. Figure 1.2 demonstrates this concept with an example of a solvable configuration [2].

Is the puzzle solvable?

12	1	10	2
7	11	4	14
5	-1	9	15
8	13	6	3

Position of blank form bottom: even row.
Number of inversions:49 inversions.
Solvable? **Yes.**

Figure1.2 solvability check explained visually.

"manhattan" is a function that runs through a nested for loop with the intend to check if each tile is in the correct position or not. If not, it calculates the absolute value of the difference in horizontal and vertical distances of the current position and the correct position the tile should be located. The return value is then the sum of all the Manhattan distances of each tile [3].Section 2 of the appendix shows the formula used in calculation.

3.2Algorithm class design:

The algorithm class was created to separate the algorithm from the rest of the code for both organization and structure of the code. In figure 1.3 the general layout of the class and what it entails in the public and private sections are illustrated.

Algorithm

```
public:
Algorithm();
Void IDA_starr ();
Int search_path();
Vector<string>  getMoves();
private:
void copyPuzzle();
Pair<int,int> findTheSpace();
//variables:
 puzzleBoard board;//object oriented
int boundary value ,int minimumcost, int
puzzleNumber , int maxMoves;
bool maxMovesExceeded;
```

Figure 1.3 Layout of the puzzle board class

3.2.1Algorithm.h declarations and deliverables:
"Algorithm.h" is the header file of the class that consists of public and private sections. The private section declares all variables and user-defined functions. The public section declares user-defined functions that can be recognized when in use in the main.cpp and the Algorithm.cpp. The sub section to follow describes the use and overall functionality of each of these variables and functions.

3.2.2 Algorithm.cpp :
The file begins with a constructor that declares the private variables "maxMoves" to the restriction of 1000 moves and declares "maxMovesExceeded" to false initially.
"getMoves" has a basic functionality of getting the moves that are stored in the vector of strings.
"copyPuzzle" runs through the rows and columns of an array through a nested for loop to make a duplicate of the previous puzzle that was used. This function plays a role in not making the same move twice (if not the best move required to achieve the shortest distance).
"findTheSpace" finds the initial position of the blank space via a nested for loop that will run through each row

and column to verify the correct index of the row and column of the blank space.
"search_path" is an essential function for implementing the IDA star algorithm. The function will find the most suitable route to take to get the least amount of moves to solve a puzzle, it does this by first determining if the puzzle is solvable or not, if solvable it will calculate an f score that adds the Manhattan distance to a move that was previously (g score) made to check the best distance ,if this f score is greater than the boundary value it will became the new boundary value, hence a recursive call is made. The minimum cost is set to a high value to ensure a move will be made initially. A move occurs according to the move functions discussed in the *puzzleBoard* class, the blank position is found for each move via the *"findTheSpace"* function and a move will be made however, the indication of these moves to the output will display the opposite index of the row and column of the blank space as it is the valued tile that will is moving in a general sense and not the blank space. If a move is not the best move that allows the shortest path to be achieved the puzzle will be copied via the *"copyPuzzle"*. The process is repeated until a suitable move is made.
"IDA_starr" is the function that was used to implement the algorithm in the main file. It works hand in hand with the search path algorithm as it initially checks the solvability and if not solvable will not continue the algorithm. If it is solvable it ensures a move is made by setting the minimum cost to an infinite number and uses the *"search_path"* function to make the most efficient move until it has been solved, however if it reaches more than 1000 moves the program will immediately end [4].

3.3 main.cpp:

The main file is the most critical part of the program as it enables all the user-defined functions in the respective classes to be put into action. This is ensured by including the respective header files of the classes in use and all other header files needed to access special functions and vectors, hence including the <vector> and <string> header files. The main function enables the reading in and out to files via the <fstream> header file and makes no use of user input or output to the console other than for debugging purposes via the <iostream> header file. The program initiates with reading puzzles from an input text file and stores the puzzles in a two-dimensional array. The classes are declared as objects, hence the program will be object orientated to achieve a successful algorithm. The moves are stored in a vector and are looped through a range-based for loop until the puzzle is solved to keep track of the moves. The IDA star algorithm is called via the object of the class and the algorithm is implemented in the program to achieve necessary output according to the specifications. Section 3 of the appendix provides two flow diagrams to show the flow of the algorithm and general program.

4.Results:

The program compiles, runs and displays an output according to the assumptions and specifications listed in section 2. The IDA* algorithm was researched to be the most efficient according to the least possible moves made and was proven true in the implementation of this program [4].
Due to the multitude of puzzles several tests could be done however, two experiments were conducted to test if the algorithm worked efficiently.100 puzzles were read through the input text file and of that 100 ,50% of the puzzles displayed the solvable output and the other 50 displayed the unsolvable output. The time for each puzzle could not be tested accurately to determine the amount of time needed for a different number of moves, thus leading to the second experiment where 10 of the solvable configurations were taken and a program run was recorded. Figure 1.4 shows the relation of the time compared to the moves needed to solve each puzzle.

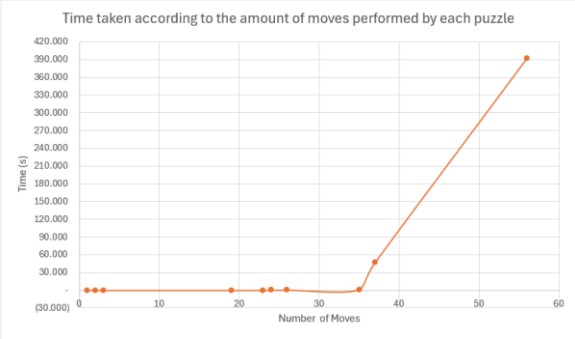

figure1.4 Relation of time taken to do a set number of moves.

In figure 1.4, the algorithm efficiency was tested, and it was observed that it takes an average of 4 seconds to solve poorly scrambled puzzles which require less than 40 moves. It takes an average time of approximately 7 minutes to solve a very well scrambled puzzle which requires more than 40 moves to be solved.

5.Critical Evaluation:

In the analysis of figure 1.4, it was observed while conducting testing that puzzles with a lower number of moves and less complexity have very fast run time and puzzles with greater complexity and moves the run time increases exponentially. Hence, as the number of moves increases the run time increases creating a proportional relation. This program only makes use of one heuristic in the IDA star algorithm to calculate the shortest path to take (the Manhattan distance). The Manhattan distance is very efficient and approximates the distance better than other heuristics [3], however, it was not enough to have a fast-running program. The specs of the computer also play a role in this run time and might be slower or faster on other consoles. The IDA star algorithm will deliver the least number of moves, but a trade-off would be the time efficiency for the complexity of puzzles. It was decided to have the correct solution in the lowest number of moves regardless of the time taken to achieve this result. Arrays were used instead of vectors to store the puzzles which comes with advantages and some disadvantages. The advantage was that it does not take up as much memory space as vectors do as they have a set size and are not dynamic, however the disadvantage was that vectors have built in functionalities and do not require passing by reference and as many user-defined functions as arrays do to complete the same task with success.
Vectors were indeed used, however, just to store the moves of the puzzle, which lead to more use of memory space. The use of classes made the game more structured and organized.

6.Recommendations:

To improve the algorithm that is implemented one can add another heuristic to the Manhattan distance. Linear conflicts will aid in increasing the efficiency of the algorithm. It gets better and more time efficient results. Linear conflicts are considering two tiles that are in the same row or column which are in a different order, but their desired position is in that row or column and the blank position is excluded from the process [5]. Another recommendation is to combine functions together to avoid redundancy such as the move functions. This could be done with structs; however, a better understanding of the concept and time would need to be put into place to implement this in the future.

7.Conclusion:

The famous ’15 puzzles’ game was designed and implemented in such a way that the success criteria were met. The code was well-structured with the use of two classes and the program runs and compiles according to the specifications of the output and has no errors, therefore it was debugged effectively. The program can solve any solvable configuration with some time constraints, in future works one can improve the implementation of the algorithm with more effective heuristics.

References

[1] "“15 puzzle - Rules and strategy of puzzle games,”," gambiter.com, 2022. [Online]. Available: https://gambiter.com/puzzle/15_puzzle.html.

[2] "“How to check if an instance of 15 puzzle is solvable?,”," GeeksforGeeks, 23 April 2016. [Online]. Available: https://www.geeksforgeeks.org/check-instance-15-puzzle-solvable/.

[3] J. L. a. X. Zhang, "“Enhanced Simplified Memory-bounded A Star (SMA*+).”," http://www.cis.umassd.edu/~x2zhang/home/pub/C2-GCAI2017-SMA.pdf, 2017.

[4] B. A. Mahafzah, "“Parallel multithreaded IDA* heuristic search: algorithm design and performance evaluation,”," *International Journal of Parallel, Emergent and Distributed Systems,* vol. vol. 26, no. no. 1, p. pp. 61–82, Feb. 2011.

[5] M. Kim, "Kim, Michael | Solving the 15 Puzzle," 27 January 2019. [Online]. Available: https://michael.kim/blog/puzzle.

Appendix:

Section 1: Time management

Table1: Actual time breakdown vs estimated time breakdown.

Activity	Actual time Breakdown(hours):	Estimated time breakdown(hours):
Background	5.5	7.5
Analysis and design	12	10
Implementation	15	10
Testing	7.5	10
Documentation	14	12.5
Total time	54	50

As can be seen from table 1, the project took 4 hours longer to complete than expected. The analysis and design, implementation and documentation took longer than expected to perfect and ensure efficiency of the code and an accurate report. The background and testing were not worked on as much as expected but the time spent was effective. If more time was spent on the background, more research could have been done on better heuristics, hence a better and more efficient algorithm could be put into action.

Section 2: Calculation of heuristic:

$$Manhattan\ distance = \sum_{i=1}^{n} |(x[i] - y[i])| = |x[2] - x[1]| + |y[2] - y[1]| + \cdots + |x[n] - x[1]| + |y[n] - y[1]|$$

Section 3: Visual Aid

Contained in this section of section of the appendix is a flow chart and visual description of the IDA* algorithm.

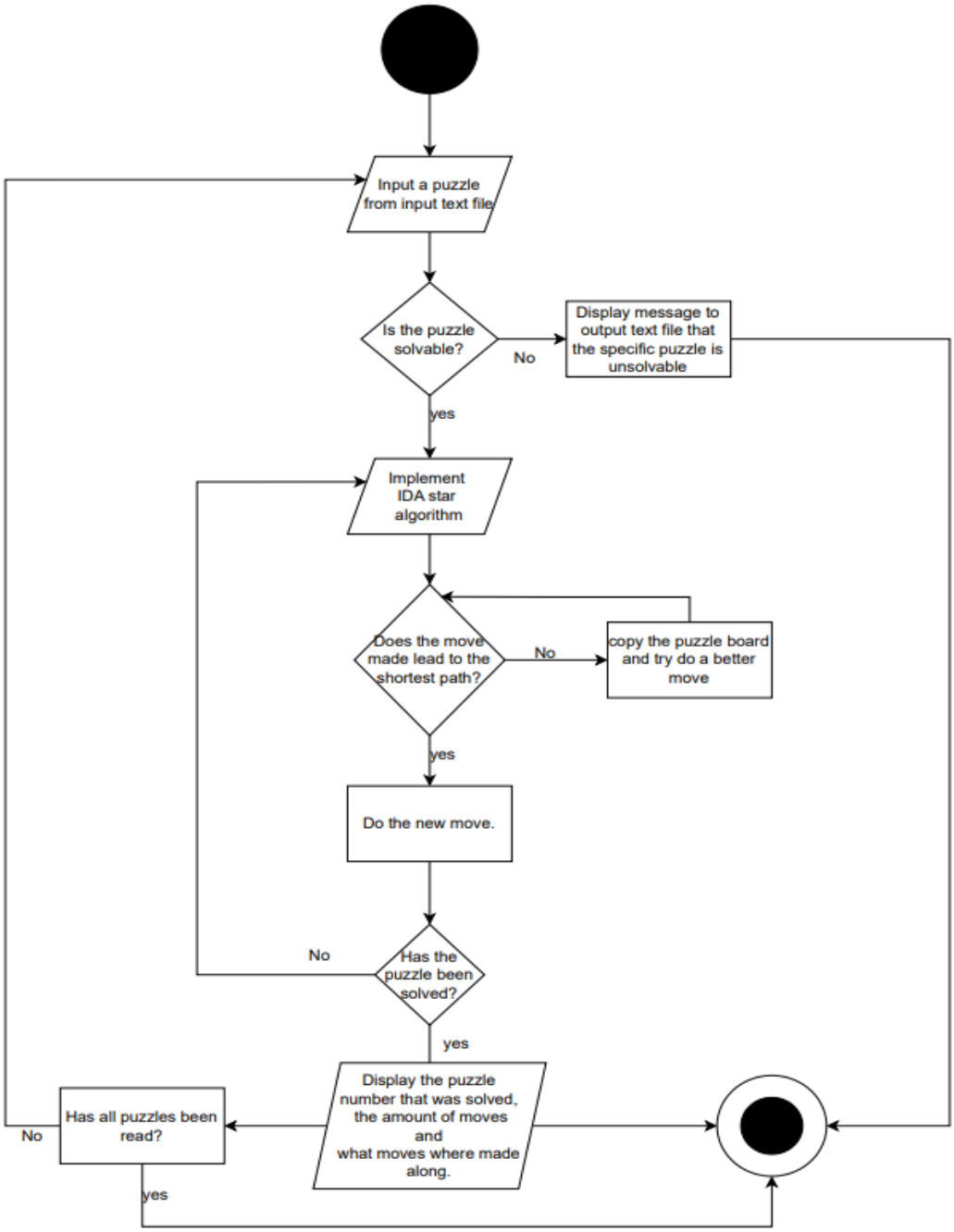

Figure 1: flow chart of main.cpp, which combines all the classes into one function to display accurate results.

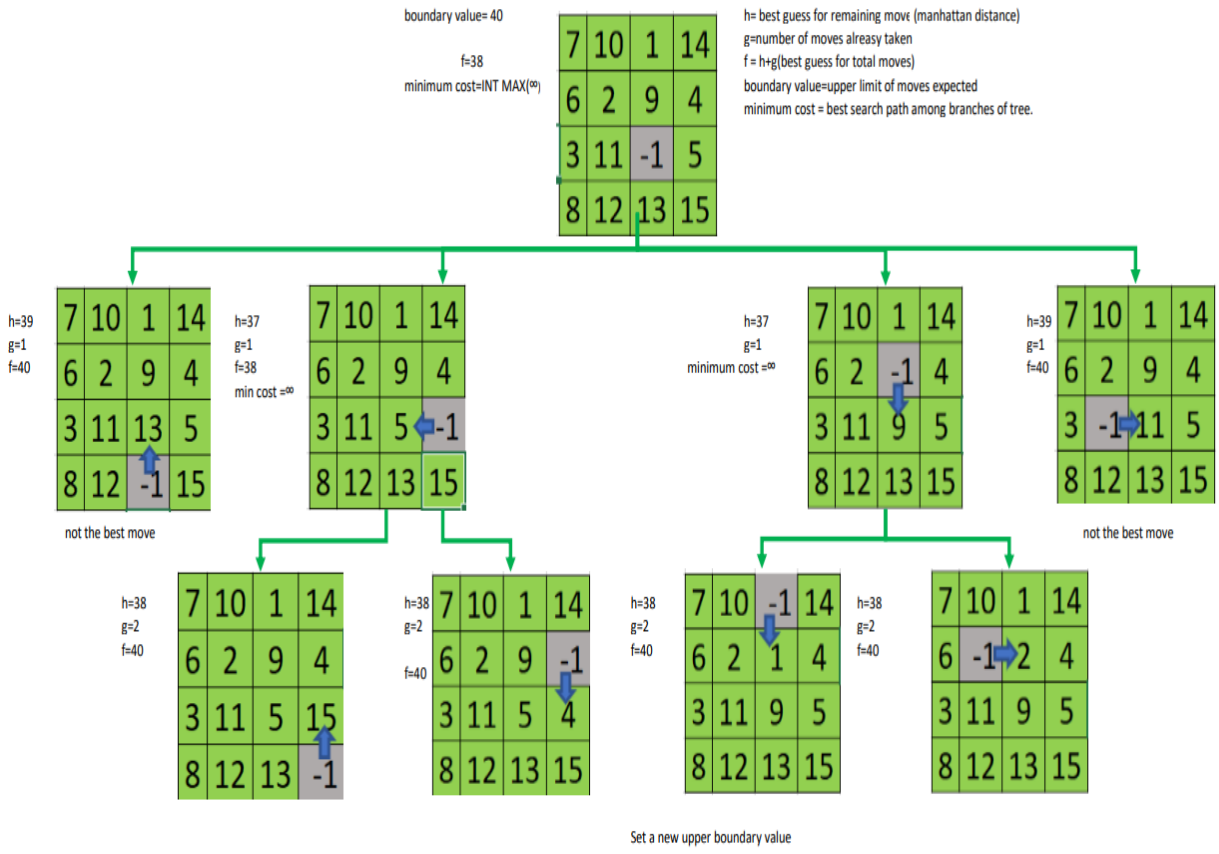

Figure 2 Flow diagram visually describing the IDA algorithm*

The IDA* algorithm can be described visually to for understanding purposes and to break down the algorithm into manageable steps hence aiding in the necessary calculations and functions needed to build the code for such a complex algorithm. Figure 2 describes and relates the algorithm to how it was implemented in the code.